CLAIMPROMOTEBENCH: SOURCE-TRACEABLE REGRESSION TESTS FOR MODEL SERVICES UNDER POLICY REVISION

Anonymous authors
Paper under double-blind review

ABSTRACT

Executable policy revisions can change downstream actions for fixed inputs, yet model-service tests rarely isolate policy version. CLAIMPROMOTEBENCH turns finite differences between deterministic policy versions into paired route tests whose oracle action must change. It records source witnesses, rule and action-transition coverage, exact minimum structural suites, parser provenance, and failure-inclusive endpoints. Six revisions across five public projects ground policy, verifier, and adapter fault classes. AGT is replayed parent-to-fix; agentgateway's parent is source-audited and its fixed commit passes 33 offline tests. Two Gatekeeper revisions yield 12 pairs and a two-pair minimum. Across 144 hosted calls, all 63 raw-strict actions match OPA, while timeouts and envelope failures reduce end-to-end pair completion to 21/36 and 0/36. An audit of 1,320 frozen records separates raw-strict parsing, registered route parsers, and full contracts; six fenced Qwen challenge outputs account for a 128/180 versus 132/180 item difference. R6 records 113/150 pairs end to end and 113/123 conditional. A commit-frozen, action-balanced challenge yields 48/90 and 13/90 full-contract pairs end to end versus 48/64 and 13/61 conditional, with `DEMOTE` recall of 4/30 and 0/30. Complete confusion matrices retain terminal failures. These results establish a source-traceable regression protocol for finite authored policy changes. Deployment prevalence and naturalistic model interpretation remain open.

1 INTRODUCTION

Executable decision policies evolve. Content-moderation studies describe frequent policy updates, limited new labels, and conflicts between historical labels and current rules (Mullick et al., 2023; Ji et al., 2026). Software-policy repositories expose the same maintenance problem in executable form. A Gatekeeper revision changed an empty FlexVolume allowlist from permitting every driver to blocking every driver; another restricted a selector search to Services (Open Policy Agent Gatekeeper Library, 2020; 2021). Trivy corrected an inverted admission predicate (Aqua Security Trivy Checks, 2024), and Cosign repaired a verifier that could accept a true `deny` result or an undefined query (Sigstore Cosign, 2022). Microsoft Agent Governance Toolkit (AGT) repaired a disagreement between Rego pattern matching and deny-verdict generation at agent lifecycle policy points (Microsoft Agent Governance Toolkit, 2026). Agentgateway corrected intervention classification and multipart-content preservation in a versioned Bedrock guardrail adapter (Agentgateway Project, 2026). These six revisions across five projects establish policy, verifier, and adapter fault classes. A fixed-query discovery audit found the latter two cases; it does not estimate their prevalence. Prior prompt-regression work holds an application task fixed while a hosted LLM API evolves (Ma et al., 2024). We examine the complementary version axis: the service route is fixed while the executable policy changes.

The evaluated model-service boundary is an author-constructed, prospective test setting. It asks whether a route presented with structured evidence and an explicit policy version follows the new decision surface and satisfies a downstream output contract. A directly callable policy executor remains the preferred enforcement mechanism for structured inputs. Here it supplies the defining oracle for testing a model-mediated interface. The study therefore establishes a reusable test construction; deployed incidence and operational benefit require separate field evidence.

Static accuracy gives an incomplete answer. A route can ignore the policy version and exploit stable field–action correlations. A response can also expose the correct action while violating a larger schema, or fail before any action is available. Valid-only scoring removes those interface failures from the denominator. Rule coverage creates another blind spot: a suite may exercise every edited rule while omitting action transitions induced by their interactions.

CLAIMPROMOTEBENCH accepts two deterministic evaluators P_0, P_1, a finite context set $\mathcal{X}$ with stable identifiers, registered changed-rule witnesses, and declared coverage inventories. Its enumerator executes both policies and retains contexts whose prescribed actions differ. It duplicates each context and varies only the active policy. A pair is correct only when both actions follow their respective policies. The suite reducer reports changed-rule and directed action-transition coverage and selects a cardinality-minimum structural-coverage suite. Policy parsing, grammar synthesis, rule-change discovery, and witness inference remain caller responsibilities. Route evaluation then separates raw-strict parsing, the frozen registered route parser, full contract validity, and exact structured-decision agreement while retaining every scheduled call in end-to-end denominators.

The evaluation has three evidence layers. Six upstream histories from Gatekeeper, Trivy, Cosign, AGT, and agentgateway establish external policy and interface fault instances. The Gatekeeper revisions then support an authored 12-pair model-service study with three frozen repetitions on two hosted model IDs. An authored four-action SLSA-inspired migration tests transition coverage, while two research-evidence suites diagnose action imbalance and endpoint sensitivity. Every model-facing mapping and label remains an author-defined instrument.

The contributions are:

1. an integrated benchmark protocol that converts finite executable policy differences into controlled policy-version pairs, relative rule/transition certificates, and a cardinality-minimum structural suite;
2. a failure-inclusive route protocol that distinguishes raw-strict, registered-parser, contract-action, and full-decision endpoints at item and pair level; and
3. a source-traceable evaluation spanning six upstream revisions in five projects, repeated hosted routes, and action-balanced evidence-state diagnostics.

The enumerator adapts established semantic-differencing and set-cover machinery. The contribution is their integration into a test object that joins an action-changing policy intervention, a paired model-service record, parser provenance, and an explicit failure-inclusive endpoint.

2 RELATED WORK

Behavioral tests, contrasts, and metamorphic relations. CheckList organizes behavioral tests by capability, and contrast sets probe local decision boundaries through controlled input changes (Ribeiro et al., 2020; Gardner et al., 2020). Metamorphic testing evaluates relations among transformed executions (Chan et al., 1998). CLAIMPROMOTEBENCH uses a policy-version intervention and an exact executable oracle; its paired score detects routes whose action remains invariant across an action-changing revision.

Conformance, semantic differencing, and regression. Labelled-transition-system and finite-state-machine methods derive conformance tests from explicit behavioral models (Tretmans, 1996; Chow, 1978). Semantic differencing identifies behavioral changes between test-design models (Tzoref-Brill & Maoz, 2017). Margrave computes witnesses for access-control policy disagreement, while Cirg uses change-impact analysis to generate and reduce policy tests (Fisler et al., 2005; Martin & Xie, 2007b). Mutation testing and suite reduction supply established adequacy machinery (Martin & Xie, 2007a; Harrold et al., 1993). Prompt regression measures service evolution under a fixed application task (Ma et al., 2024). CLAIMPROMOTEBENCH combines these ideas at a model-service boundary: a fixed route receives paired policy versions, rule and action-transition coverage are minimized jointly, and every service failure stays in the scored denominator. The source-traceable integration is the method contribution.

Table 1: Positioning by controlled coordinate. CLAIMPROMOTEBENCH contributes the joint setting in the final row; each constituent technique is established.

Test family	Controlled coordinate	Primary oracle or endpoint	Unjoined coordinate
Behavioral / contrast	input perturbation	labels or behavioral invariants	executable policy revision
Conformance / policy diff	specification or policy	executable semantics	hosted model-service record
Rule / specification following	textual rules	task labels	executable version difference
Structured output / abstention	schema or coverage	syntax, contract, or risk	action-changing policy pair
Prompt regression	hosted service time	application tests	policy version
This work	policy source	executable action plus route contract	source-to-route regression

Rule and specification following. RuleArena and SpecEval test rule-governed and specification-following behavior (Zhou et al., 2025; Ahmed et al., 2025). GuideBench includes guideline updates, and TDAD studies specification evolution with hidden tests and semantic mutants (Diao et al., 2025; Rehan, 2026). Policy-to-Tests translates governance prose into executable rules (Datla et al., 2025). CLAIMPROMOTEBENCH begins with two deterministic evaluators and treats policy version as the controlled coordinate. Its scope covers finite behavioral differences after policy authoring, context construction, and witness registration.

Structured output and selective prediction. Grammar-constrained decoding and incremental parsing improve syntactic validity (Scholak et al., 2021; Geng et al., 2023); JSONSchemaBench and BFCL evaluate schemas and tool arguments (Geng et al., 2025; Patil et al., 2025). Selective prediction evaluates risk under a learned reject option (Geifman & El-Yaniv, 2017; 2019). Our action set contains an explicit `HOLD` decision. Transport, syntax, schema, and semantic failures occupy a separate terminal category and remain errors end to end.

Research agents. ScienceAgentBench, CORE-Bench, and POPPER evaluate data analysis, reproducibility, and sequential falsification (Chen et al., 2025; Siegel et al., 2025; Huang et al., 2025). The auxiliary research-evidence ontology covers one downstream state transition after evidence registration. Evidence discovery and end-to-end scientific work remain outside its scope.

3 METHOD

3.1 FINITE POLICY-DIFFERENCE ENUMERATION

The caller supplies deterministic evaluators P_0 and P_1, a finite set $\mathcal{X}$ of complete typed contexts with stable identifiers, registered changed-rule witnesses, and the rule inventory to cover. Let $\pi_P(c)$ denote the prescribed action. The enumerator executes both evaluators and defines

$$\Delta(P_0, P_1; \mathcal{X}) = \{c \in \mathcal{X} : \pi_{P_0}(c) \neq \pi_{P_1}(c)\}.$$

Each $c \in \Delta$ yields a pair $((P_0, c), (P_1, c))$. The context, query, and output contract are identical; only the policy source changes. This construction makes policy-version sensitivity a necessary condition for pair correctness. A deterministic route invariant to the policy returns the same action twice, so at most one member can be correct and pair accuracy is zero.

The enumerator associates every candidate with its registered changed-rule witnesses and directed action edge $(\pi_{P_0}(c), \pi_{P_1}(c))$. For suite T, adequacy is

$$A(T) = \left(\frac{|R(T)|}{|R_\Delta|}, \frac{|E(T)|}{|E_{\text{reach}}|} \right),$$

where R_Δ is the registered changed-rule inventory and E_{reach} is the action-edge set observed by exhaustive execution over $\mathcal{X}$. Both certificates are relative to the published context set and caller-declared witnesses. Their validity depends on context completeness and witness correctness.

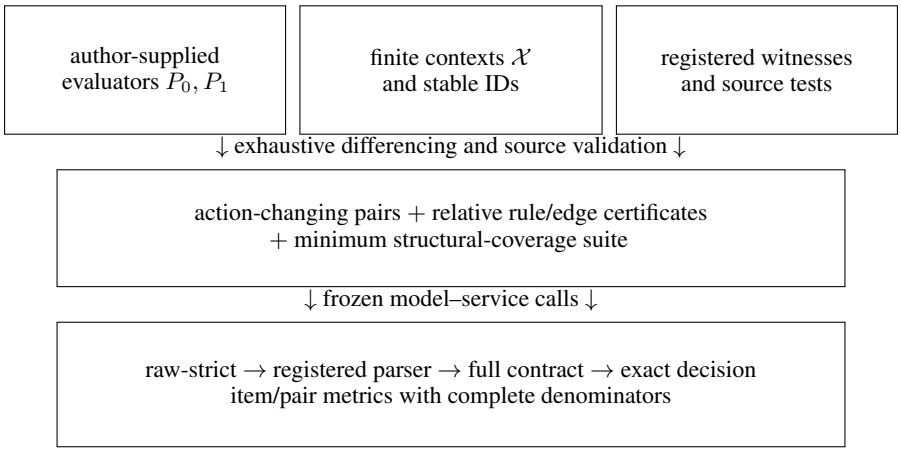


Figure 1: Benchmark and evidence flow. Caller-supplied artifacts precede enumeration; source validation and suite construction finish before model calls. Each scoring layer corresponds to a distinct interface requirement.

Selecting the smallest structural-coverage suite with $A(T) = (1, 1)$ is a set-cover instance. We use a standard exact bit-mask dynamic program with $O(n2^{r+k})$ state updates for n candidates, r rules, and k edges. A specialized search selects one context per edge while tracking rule masks; a complete k-pair witness meets the edge lower bound and is globally minimum for the candidate pool. These solvers are checkable implementation components. Appendix A gives the proof and optimality certificate.

3.2 Parser and contract endpoints

For scheduled item x, let $y(x) = \pi_P(c)$; let $r(x)$ be the action parsed directly from retained raw text; let $p(x)$ be the action recorded by the route's frozen registered parser; let $v(x)$ indicate full output-contract validity; and let $d(x)$ be the parsed structured decision. The raw-strict parser accepts exactly one duplicate-free top-level JSON object and performs no fence or substring extraction, repair, coercion, or non-standard constant handling. Registered parsers retain their historical route-specific behavior. Their provenance distinguishes direct parsing from deterministic single-object extraction. No audited response contains a duplicate key. We report:

$$
\begin{aligned}
C_R(x) &= \mathbb{1}[r(x) = y(x)], && \text{raw-strict action,}\\
C_P(x) &= \mathbb{1}[p(x) = y(x)], && \text{registered-parser action,}\\
C_C(x) &= \mathbb{1}[v(x) = 1]\mathbb{1}[p(x) = y(x)], && \text{contract action,}\\
C_F(x) &= \mathbb{1}[v(x) = 1]\mathbb{1}[d(x) = d^*(x)], && \text{full decision.}
\end{aligned}
$$

The task-specific contract contains five fields for Gatekeeper and six fields for the evidence-state tasks. Pair correctness is the product of the two member indicators for the selected endpoint. End-to-end metrics divide by every scheduled item or pair. Conditional action metrics divide by available parser actions or contract-valid items and complete pairs; they describe behavior after the indicated interface succeeds. Terminal failures are shown as `N` in confusion matrices, outside the policy action set. Response contents from failed cells are omitted.

4 Evidence and Evaluation Design

4.1 Source-traceable revisions

Table 2 separates policy revisions, verifier faults, authored stress instruments, and auxiliary robustness tests. The Gatekeeper study preserves exact old and new Rego source bytes, their parent and change commits, upstream regression-test names, and Apache-2.0 notices. OPA 1.19.1 reproduces both changes. We author a finite 12-context grammar around the upstream tests: six contexts for the FlexVolume empty-list change and six for the selector-kind change. Mapping zero violations to

Table 2: Evidence hierarchy. Structural minima are exact relative to each committed candidate set.

Instrument	Revision or instrument	Evaluation role	Size	Coverage or check
Source-anchored revisions and faults				
Gatekeeper	two upstream Rego revisions	central authored route test	12 pairs	2 rules, 2/2 edges; min. 2
Trivy KCV0030	upstream predicate fix	policy-fault reproduction	4 cases	2/2 edges; min. 2
Cosign Rego verifier	upstream query fix	interface-fault reproduction	6 cases	2/2 edges; min. 4
AGT helper	upstream verdict fix	agent-policy interface fault	2 cases	1/1 edge; min. 1
agentgateway	Bedrock adapter fix	guardrail-interface fault	33 tests	33/33 fixed
Author-defined structural stress instrument				
Authored SLSA	12-rule policy migration	four-action transition test	60 pairs	12 rules, 10/10 edges; min. 10
Auxiliary evidence-state diagnostics				
Research r6	authored paired contrasts	coverage addendum	150 pairs	–
Public challenge	commit-frozen authored contrasts	balanced robustness	90 pairs	–

ALLOW and one or more violations to HOLD is an author policy mapping. The resulting 24 items were frozen before any route call. All labels are deterministic author mappings; the study contains zero human annotations.

The Trivy reproduction executes four upstream regression cases around the corrected inverted predicate. The Cosign reproduction exercises six cases and four verifier fault modes using the historical Go query logic and upstream tests. Cosign changes the verifier–policy interface and provides a distinct boundary-fault evidence class.

A frozen fixed-query discovery audit screened 465 unique public repository seeds and reviewed 11 joint model/policy candidates at source level. Inclusion required a public model or agent path, an executable or versioned policy boundary, a finite downstream action, a historical fault, a usable license, and auditable source evidence. Two candidates met every criterion. The AGT reproduction evaluates exact parent and fixed Rego bytes; its two action labels are an author mapping of an undefined deny verdict to the upstream-documented caller fallback. The agentgateway check compiles the exact change commit and runs its offline Bedrock guardrail tests. The discovery process is neither random nor exhaustive.

4.2 Route studies and auxiliary instruments

The central route study repeats all 24 Gatekeeper items three times on each of two hosted routes, for 144 scheduled calls. Each cell receives one request, zero client retries, and a 180-second timeout. The complete schedule, prompts, route labels, endpoint definitions, and no-retry rule were committed before the first call. The routes use two model IDs through the same provider, base URL, OpenAI-compatible API, prompt-only output mode, and adapter settings. Provider-side serving internals are opaque. Their results are descriptive route measurements.

The authored SLSA instrument defines two profiles over structured attestation fields and assigns one of four actions: ALLOW, HOLD, DEMOTE, or ESCALATE (SLSA Steering Committee, 2026; in-toto Attestation Maintainers, 2026). Its rule-stratified suite covers 12 changed rules. Its graph suite covers every reachable directed action edge. Two pinned four-bit local routes provide a stress test of the enumerated pairs. The deterministic executor supplies the labels and is the *definitional policy ceiling*. It is an oracle, while learned rows remain empirical controls. Registered structured random-forest controls test profile sensitivity directly: the policy-aware control reaches 107/120 items and 47/60 pairs end to end, while the profile-blind control reaches 57/120 items and 0/60 pairs. These rows serve as diagnostic controls for the paired endpoint.

The research-evidence ontology supplies two auxiliary suites. R6 contains 300 items with only eight DEMOTE and eight ESCALATE labels. The challenge contains 90 pairs, 180 items, and 15 factors; it balances 90 ALLOW items against 30 each of HOLD, DEMOTE, and ESCALATE. Its cases were author-created and commit-frozen before calls according to retained workflow receipts. It therefore

measures robustness on known cases rather than hidden-test generalization. A multinomial Naive Bayes sanity control is deliberately weak: on r6 it reaches 184/300 items and 34/150 pairs end to end, with balanced action recall 0.328. Its role is a weak shortcut check.

5 RESULTS

5.1 SOURCE VALIDATION AND FINITE STRUCTURAL COVERAGE

OPA execution reproduces both Gatekeeper behavior changes on all 12 contexts. The suite covers 2/2 changed rules and both `ALLOW→HOLD` and `HOLD→ALLOW` transitions. One pair realizes one edge, and a two-pair witness also covers both rules; the witness meets the two-edge lower bound. The Trivy cases reproduce two transitions in each direction around the inverted predicate, with a two-case minimum. The Cosign cases reproduce four documented verifier fault modes and both directions; four cases are required to cover the four modes. Against the exact AGT parent source, 11/15 post-fix upstream tests pass; all 15 pass after the fix. Both decision cases change from an undefined deny verdict to `deny`, yielding two author-mapped `ALLOW→HOLD` transitions and a one-case minimum. At the exact agentgateway change commit, all 33 selected offline tests pass. The source audit traces the old adapter's overbroad anonymization classification and multipart-content replacement; it does not retrofit the post-fix test module onto the parent commit. These results establish concrete policy, verifier, and guardrail-interface fault instances.

The authored four-action migration exposes a separate structural result. Its 12-rule stratified suite covers 12/12 changed rules and 6/10 reachable edges. The graph suite covers 12/12 rules and 10/10 edges. An exact search selects a 10-pair, 20-call joint suite, reducing the 120-call graph suite by 83.3%. The two local routes score 10/60 and 2/60 pairs on the full graph. This minimum certifies coverage of the declared rules and observed edges. Stochastic fault detection and release sufficiency remain unmeasured. The registered policy-aware random-forest control reaches 107/120 items and 47/60 pairs, while the profile-blind control reaches 57/120 items and 0/60 pairs. The zero-pair result confirms that the pair endpoint detects policy-profile invariance on this authored graph.

5.2 REPEATED ROUTE BEHAVIOR ON UPSTREAM REVISIONS

The central hosted result is an exact-interface completion gap. Table 3 reports every scheduled Gatekeeper cell. Raw-strict and contract-action scores coincide because every raw-strict object also satisfies the task contract. The DeepSeek route records 15/24, 19/24, and 19/24 actions and totals 53/72 items and 21/36 pairs; 52/72 decisions and 20/36 pairs are fully exact. The Qwen route totals 10/72 items and 0/36 pairs. All 63 raw-strict actions match the OPA oracle, so this study observes zero semantic action errors after raw-strict extraction. Conditional item and pair accuracy is 1.0 whenever the required item or complete pair is available.

Terminal metadata identifies the limiting interface behavior; response content remains excluded. The Qwen route produces 62 Markdown-fenced objects, all rejected by the frozen strict parser. The DeepSeek route records 12 read timeouts, three fenced objects, three non-JSON prefixes or envelopes, and one length-terminated non-JSON response. The protocol performs zero response repair or rescoring. Across repeated cells, 13/24 DeepSeek items are raw-strict correct in all three repetitions, compared with 0/24 for Qwen; 5/12 DeepSeek pairs pass all three repetitions.

5.3 ENDPOINT SENSITIVITY AND EVIDENCE-STATE DIAGNOSTICS

The offline endpoint audit covers 1,320 previously scheduled items across ten route stacks. Raw-strict parsing finds 69 extractable actions outside the full six-field contract; 49 actions and 40 pairs are correct. Conversely, 452 contract-valid registered-parser actions are unavailable to raw-strict parsing, including 329 correct actions and 78 correct pairs. Most of the latter occur on historical local routes whose registered parsers used deterministic single-object extraction. The endpoints are nonnested and answer different integration questions.

On the fixed SLSA Qwen route, raw-strict action yields 120/120 items and 60/60 pairs, while the full contract yields 88/120 and 28/60. On the balanced Qwen challenge, the registered parser accepts six fenced objects—four correct actions and two correct pairs—that raw-strict parsing rejects. Raw-strict

Table 3: Gatekeeper route results by repetition. All columns are end to end. R is raw-strict action; F is exact full decision. Contract action equals R for every row.

Route	Repetition	R item	F item	R pair	F pair
Qwen	1	3/24	3/24	0/12	0/12
Qwen	2	2/24	2/24	0/12	0/12
Qwen	3	5/24	5/24	0/12	0/12
	total	10/72	10/72	0/36	0/36
DeepSeek Flash	1	15/24	15/24	6/12	6/12
DeepSeek Flash	2	19/24	18/24	7/12	6/12
DeepSeek Flash	3	19/24	19/24	8/12	8/12
	total	53/72	52/72	21/36	20/36

Table 4: Auxiliary item/pair metrics. All raw and contract scores are end to end; the last column is conditional on contract validity. "–" marks unavailable retained raw text for r6.

Route	Reg. valid	Raw item/pair	Contract item/pair	Cond. item/pair
r6 Qwen	272/300	–	258/300; 113/150	258/272; 113/123
Challenge Qwen	148/180	128/180; 46/90	132/180; 48/90	132/148; 48/64
Challenge Flash	146/180	105/180; 19/90	90/180; 13/90	90/146; 13/61
SLSA Qwen	88/120	120/120; 60/60	88/120; 28/60	88/88; 28/28
SLSA Flash	108/120	110/120; 51/60	108/120; 49/60	108/108; 49/49

scores are therefore 128/180 items and 46/90 pairs, compared with 132/180 and 48/90 under the full contract. The balanced Flash route changes from 105/180 and 19/90 under raw-strict parsing to 90/180 and 13/90 under the contract. Registered-parser actions and full-contract actions coincide in these frozen records because a parser-owned decision is retained only after contract validation.

Action support changes the diagnosis. Table 5 gives end-to-end contract-action recall. R6 Qwen has only eight examples for each rare action and recovers 0/8 `DEMOTE` and 5/8 `ESCALATE`. The balanced challenge increases each non-`ALLOW` action to 30 cases: Qwen recovers 4/30 `DEMOTE`, and Flash recovers 0/30. The complete confusion matrices in Appendix C retain terminal failures in every row.

6 Discussion

The external histories validate a composed regression target. The six revisions show that fixed inputs can change action because of policy source, verifier query semantics, or guardrail-adapter logic. The AGT and agentgateway cases place the latter two failure classes directly at public agent or model boundaries. A regression test that isolates only the model or only the policy misses this composition. ClaimPromoteBench covers the policy-version coordinate by joining executable differences to paired route records and failure-inclusive endpoints. The discovery audit establishes examples, while their deployment frequency remains unknown.

The hosted finding concerns exact-interface completion. The two Gatekeeper routes complete 21/36 and 0/36 raw-strict pairs, while all 63 available actions agree with OPA. The recorded gap arises from 12 service timeouts and 69 serialization or envelope violations. The 1,320-item audit shows why parser semantics must be named: raw-strict parsing finds correct actions outside the full contract, whereas registered parsers recover other actions through their frozen extraction policy. The six fenced Qwen challenge outputs change the raw-strict result to 128/180 items and 46/90 pairs, below its 132/180 and 48/90 full-contract result. Neither endpoint dominates. Interface consumers should select the endpoint that matches their contract while preserving the scheduled denominator and parser provenance.

Pair and action diagnostics expose distinct shortcuts. The policy-aware control reaches 47/60 pairs, whereas the profile-blind control reaches 0/60, showing that paired scoring detects policy

Table 5: End-to-end contract-action recall by gold action.

Route	ALLOW	HOLD	DEMOTE	ESCALATE
r6 Qwen	143/150	110/134	0/8	5/8
Challenge Qwen	79/90	26/30	4/30	23/30
Challenge Flash	68/90	9/30	0/30	13/30

invariance on the authored transition graph. R6's 113/150 end-to-end versus 113/123 conditional pairs accompany 0/8 DEMOTE recall. Balancing the challenge exposes the same failure with more support: Qwen and Flash reach only 4/30 and 0/30 on DEMOTE, while their full-contract pair results are 48/90 and 13/90 end to end versus 48/64 and 13/61 conditional. Complete confusion matrices retain terminal failures as N, preventing aggregate accuracy from concealing action-specific and interface failures.

The intended use is prospective and bounded. The evaluated model-service boundary is an authored test construction for policy maintenance. A direct deterministic executor remains the preferred enforcement path for fully structured inputs and defines the policy ceiling here. The r6 Naive Bayes control reaches only 184/300 items and 34/150 pairs, so it supplies a weak shortcut check. Field evidence must establish whether the model-mediated boundary occurs often enough to justify operational adoption.

7 LIMITATIONS

The model-service study is prospective and synthetic. It covers two Gatekeeper revisions, one author-built context set, two hosted model IDs, and three repetitions. Trivy, Cosign, AGT, and agentgateway broaden source validity; route replication covers only Gatekeeper. The authored SLSA migration broadens the action graph while preserving synthetic policy scope. Population estimates for policies, model families, deployments, and operational impact remain open.

The Gatekeeper, AGT, SLSA, and research-evidence action labels are author policy mappings. The study contains zero human annotations. The fixed-query external audit screened 465 repository seeds and source-reviewed 11 candidates, but it is not a random or exhaustive sample. Its two qualifying cases validate boundary faults, and neither asks a model to interpret a changed executable policy. Naturalistic model-policy interpretation therefore remains unvalidated.

The finite context set bounds every coverage claim. Context omissions can hide behavior, and incorrect registered witnesses can invalidate rule coverage. Minimum-suite optimality certifies structural coverage of the declared rule/edge universe; stochastic fault-detection power and release sufficiency remain unmeasured. The Trivy reproduction uses a documented compatibility helper to execute historical Rego under the current runtime while preserving upstream policy bytes. The Cosign study targets verifier logic, a separate evidence class from policy-source differencing.

Hosted route results combine model weights, prompts, output constraints, provider service state, and time. Both central model IDs use the same provider, base URL, OpenAI-compatible API, prompt-only mode, and adapter settings; provider-side serving internals remain opaque. The three repetitions describe one fixed schedule and yield descriptive route measurements. The historical r6 Flash lane used an obsolete frozen service interface: all 300 requests returned HTTP 401 before model output. Later Flash calls establish executability only for their newer service paths. The service and output stack changed, so the 401 lane provides no causal comparison.

The challenge was author-created and commit-frozen before calls according to retained workflow receipts; it shares an ontology with r6 and measures robustness on known cases. Its Qwen and Flash full-contract pair scores are 48/90 and 13/90 end to end versus 48/64 and 13/61 conditional; r6 Qwen reaches 113/150 versus 113/123. The challenge's 4/30 and 0/30 DEMOTE recalls and complete confusion matrices retain every terminal failure. Six Qwen outputs are accepted by the frozen registered parser through deterministic single-object extraction but rejected by raw-strict parsing. Failed response contents remain excluded. Policy retrieval, ambiguous source text, evidence extraction, and downstream utility remain outside the evaluation.

8 CONCLUSION

CLAIMPROMOTEBENCH joins finite executable policy differences to paired model-service records, relative rule/transition certificates, an exact minimum structural suite, and parser-provenance-aware endpoints. Six public revision histories ground policy, verifier, and guardrail-interface fault classes. The Gatekeeper study exposes exact-interface completion; the r6 and commit-frozen balanced challenge expose the gap between end-to-end and conditional performance, including `DEMOTE` recalls of 4/30 and 0/30 under balanced support. Complete confusion matrices preserve terminal failures. A deterministic executor remains the definitional policy ceiling. The next discriminating test is a source-traceable deployed route in which a model interprets a changed policy across independent service stacks.

9 REPRODUCIBILITY STATEMENT

The artifact binds released policies, datasets, prompts, source bytes and tests, finite grammars, frozen schedules, route configurations, parser source, and the external discovery registry. It also binds terminal outcome classes, analyses, and privacy receipts. From the anonymous bundle root, run `python scripts/run_review_checks.py`. Validators recompute source bindings, coverage, minimum suites, endpoint metrics, parser provenance, action recalls, and confusion matrices. This is deterministic source and score replay; hosted calls are not reissued. The anonymous derivative excludes raw failed responses, free-form errors, provider identifiers, hidden reasoning, account identity, and private commit identity. Its manifest derives file counts from the generated tree. Completed one-shot studies remain frozen.

10 AI ASSISTANCE DISCLOSURE

Generative AI tools assisted literature search, protocol critique, implementation, debugging, deterministic reaggregation, source checks, and editing. The author selected the research question, policies, protocols, exclusions, analyses, and claims; inspected primary sources and rendered artifacts; verified generated code through tests and replay; and assumes responsibility. AI systems are excluded from authorship and human-annotation counts.

REFERENCES

Agentgateway Project. Bedrock guardrail mutates in place while preserving original structure. Git commit 82ab009fbb5996716e72a1fd5e5c4d4a13fa30eb, 2026. URL https://github.com/agentgateway/agentgateway/commit/82ab009fbb5996716e72a1fd5e5c4d4a13fa30eb.

Ahmed Ahmed, Kevin Klyman, Yi Zeng, Sanmi Koyejo, and Percy Liang. SpecEval: Evaluating model adherence to behavior specifications, 2025. URL https://arxiv.org/abs/2509.02464.

Aqua Security Trivy Checks. Invert logic of AVD-KCV-0030. Git commit e4defae1e4c6e346104f41e70489503069ad1b90, 2024. URL https://github.com/aquasecurity/trivy-checks/commit/e4defae1e4c6e346104f41e70489503069ad1b90.

F. T. Chan, T. Y. Chen, Shing Chi Cheung, M. F. Lau, and S. M. Yiu. Application of metamorphic testing in numerical analysis. In *Proceedings of the IASTED International Conference on Software Engineering*, 1998. URL https://hdl.handle.net/1783.1/70576.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=6z4YKr0GK6.

Tsun S. Chow. Testing software design modeled by finite-state machines. *IEEE Transactions on Software Engineering*, SE-4(3):178–187, 1978. doi: 10.1109/TSE.1978.231496. URL https://doi.org/10.1109/TSE.1978.231496.

Gautam Varma Datla, Anudeep Vurity, Tejaswani Dash, Tazeem Ahmad, Mohd Adnan, and Saima Rafi. Executable governance for AI: Translating policies into rules using LLMs, 2025. URL https://arxiv.org/abs/2512.04408. AAAI 2026 AI Governance Workshop.

Lingxiao Diao, Xinyue Xu, Wanxuan Sun, Cheng Yang, and Zhuosheng Zhang. GuideBench: Benchmarking domain-oriented guideline following for LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11361–11399. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.557. URL https://aclanthology.org/2025.acl-long.557/.

Kathi Fisler, Shriram Krishnamurthi, Leo A. Meyerovich, and Michael Carl Tschantz. Verification and change-impact analysis of access-control policies. In *Proceedings of the 27th International Conference on Software Engineering*, pp. 196–205. ACM, 2005. doi: 10.1145/1062455.1062502. URL https://doi.org/10.1145/1062455.1062502.

Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, Nitish Gupta, Hannaneh Hajishirzi, Gabriel Ilharco, Daniel Khashabi, Kevin Lin, Jiangming Liu, Nelson F. Liu, Phoebe Mulcaire, Qiang Ning, Sameer Singh, Noah A. Smith, Sanjay Subramanian, Reut Tsarfaty, Eric Wallace, Ally Zhang, and Ben Zhou. Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1307–1323. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.117. URL https://aclanthology.org/2020.findings-emnlp.117/.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Yonatan Geifman and Ran El-Yaniv. SelectiveNet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2151–2159. PMLR, 2019. URL https://proceedings.mlr.press/v97/geifman19a.html.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10932–10952. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.674. URL https://aclanthology.org/2023.emnlp-main.674/.

Saibo Geng, Hudson Cooper, Michal Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. *arXiv preprint arXiv:2501.10868*, 2025. doi: 10.48550/arXiv.2501.10868. URL https://arxiv.org/abs/2501.10868.

Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology*, 2(3):270–285, 1993. doi: 10.1145/152388.152391. URL https://doi.org/10.1145/152388.152391.

Kexin Huang, Ying Jin, Ryan Li, Michael Y. Li, Emmanuel Candes, and Jure Leskovec. Automated hypothesis validation with agentic sequential falsifications. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 25372–25437. PMLR, 2025. URL https://proceedings.mlr.press/v267/huang25n.html.

in-toto Attestation Maintainers. *in-toto Attestation Framework Specification, Version 1.2*. Cloud Native Computing Foundation, 2026. URL https://github.com/in-toto/attestation/tree/v1.2.0.

Deyi Ji, Junyu Lu, Xuanyi Liu, Liqun Liu, Hailong Zhang, Peng Shu, Huan Yu, Jie Jiang, Tianrun Chen, and Lanyun Zhu. ARGUS: Policy-adaptive ad governance via evolving reinforcement with adversarial umpiring. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 99–112. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-industry.8. URL https://aclanthology.org/2026.acl-industry.8/.

Wanqin Ma, Chenyang Yang, and Christian Kästner. (why) is my prompt getting worse? rethinking regression testing for evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering–Software Engineering for AI*, pp. 166–171. Association for Computing Machinery, 2024. doi: 10.1145/3644815.3644950. URL https://doi.org/10.1145/3644815.3644950.

Evan Martin and Tao Xie. A fault model and mutation testing of access control policies. In *Proceedings of the 16th International Conference on World Wide Web*, pp. 667–676. ACM, 2007a. doi: 10.1145/1242572.1242663. URL https://doi.org/10.1145/1242572.1242663.

Evan Martin and Tao Xie. Automated test generation for access control policies via change-impact analysis. In *Third International Workshop on Software Engineering for Secure Systems*, pp. 5. IEEE Computer Society, 2007b. doi: 10.1109/SESS.2007.5. URL https://doi.org/10.1109/SESS.2007.5.

Microsoft Agent Governance Toolkit. Bind every intervention point an adapter evaluates. Git commit e87b74530e1cbae54d5e90bc1b45ecf5c8911b28, 2026. URL https://github.com/microsoft/agent-governance-toolkit/commit/e87b74530e1cbae54d5e90bc1b45ecf5c8911b28.

Sankha Subhra Mullick, Mohan Bhambhani, Suhit Sinha, Akshat Mathur, Somya Gupta, and Jidnya Shah. Content moderation for evolving policies using binary question answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pp. 561–573. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-industry.54. URL https://aclanthology.org/2023.acl-industry.54/.

Open Policy Agent Gatekeeper Library. Update FlexVolume Drivers PSP to block all when given an empty list. Git commit 102bd7704077cb8f22967b2a75aba2aed4046b02, 2020. URL https://github.com/open-policy-agent/gatekeeper-library/commit/102bd7704077cb8f22967b2a75aba2aed4046b02.

Open Policy Agent Gatekeeper Library. Fix unique service selector behavior when a service selector is missing. Git commit 749ebacafb023738aa95ce468b0503a88d3e2d75, 2021. URL https://github.com/open-policy-agent/gatekeeper-library/commit/749ebacafb023738aa95ce468b0503a88d3e2d75.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Tzafrir Rehan. Test-driven AI agent definition (TDAD): Compiling tool-using agents from behavioral specifications, 2026. URL https://arxiv.org/abs/2603.08806.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL https://aclanthology.org/2020.acl-main.442/.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.779. URL https://aclanthology.org/2021.emnlp-main.779/.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Transactions on Machine Learning Research*, 2025. URL https://openreview.net/forum?id=BsMMc4MEGS.

Sigstore Cosign. Fix handling of policy in verify-attestation. Git commit 3b597b999ad083afd2ce0dba68fa2715ddec0f06, 2022. URL https://github.com/sigstore/cosign/commit/3b597b999ad083afd2ce0dba68fa2715ddec0f06.

SLSA Steering Committee. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2*. Open Source Security Foundation, 2026. URL https://slsa.dev/spec/v1.2/. Build provenance, artifact verification, and verification summary specifications.

Jan Tretmans. Conformance testing with labelled transition systems: Implementation relations and test generation. *Computer Networks and ISDN Systems*, 29(1):49–79, 1996. doi: 10.1016/S0169-7552(96)00017-7. URL https://doi.org/10.1016/S0169-7552(96)00017-7.

Rachel Tzoref-Brill and Shahar Maoz. Syntactic and semantic differencing for combinatorial models of test designs. In *Proceedings of the 39th International Conference on Software Engineering*, pp. 621–631. IEEE/ACM, 2017. doi: 10.1109/ICSE.2017.63. URL https://doi.org/10.1109/ICSE.2017.63.

Ruiwen Zhou, Wenyue Hua, Liangming Pan, Sitao Cheng, Xiaobao Wu, En Yu, and William Yang Wang. RuleArena: A benchmark for rule-guided reasoning with LLMs in real-world scenarios. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 550–572. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.27. URL https://aclanthology.org/2025.acl-long.27/.

A Enumerator and Reducer Guarantees

Minimum-suite hardness. Take a Set Cover instance with universe U and subsets $S_1, \ldots, S_n$. Create one required rule for each element of U and one candidate context for each S_i whose rule-witness set is S_i. Assign every candidate the same required action edge. Any nonempty selection covers that edge, and it covers all required rules exactly when the corresponding subsets cover U. A minimum joint suite therefore gives a minimum set cover.

Exact dynamic program. Index the $r + k$ required rules and edges and encode coverage as a bit mask. After processing the first j candidates, retain for every reachable mask the shortest subset, breaking ties by context identifier. Excluding candidate $j + 1$ preserves each state; including it unions its coverage mask with each prior mask. Induction on j proves that the final full-mask representative is cardinality-minimum. The implementation matches exhaustive enumeration on 64 deterministic small candidate pools.

Edge lower bound. Each pair realizes one directed edge. Covering k edges requires at least k pairs. If the specialized search selects one candidate per edge and also covers every rule, its k-pair witness meets the lower bound. This certifies the two-pair Gatekeeper suite and the ten-pair authored SLSA suite relative to their committed candidate pools.

B Failure Taxonomy and Scope Notes

The repeated Gatekeeper study classifies failures from terminal protocol metadata only. Qwen has 62 `MARKDOWN_CODE_FENCE` failures. DeepSeek Flash has 12 `READ_TIMEOUT`, three `MARKDOWN_CODE_FENCE`, three `NON_JSON_PREFIX_OR_ENVELOPE`, and one `LENGTH_TERMINATED_NON_JSON` failure. The diagnostic reads only terminal metadata. Gold labels, raw response text, hidden reasoning, free-form error strings, and provider response identifiers remain excluded.

The historical r6 Flash lane used an obsolete service interface. All 300 requests returned HTTP 401 before model output, leaving conditional metrics undefined. Later Flash routes changed the service and output stack; their successful calls establish only that those newer paths execute.

C Complete Action-Level Results

Table 6: End-to-end action recall for every route summarized in the main text. "–" marks actions outside a task's label set.

Configuration and endpoint	ALLOW	HOLD	DEMOTE	ESCALATE
Gatekeeper Qwen, raw-strict	4/36	6/36	–	–
Gatekeeper Flash, raw-strict	32/36	21/36	–	–
r6 Qwen, contract	143/150	110/134	0/8	5/8
Challenge Qwen, raw-strict	77/90	26/30	3/30	22/30
Challenge Qwen, contract	79/90	26/30	4/30	23/30
Challenge Flash, raw-strict	68/90	10/30	2/30	25/30
Challenge Flash, contract	68/90	9/30	0/30	13/30
SLSA Qwen, raw-strict	60/60	20/20	20/20	20/20
SLSA Qwen, contract	60/60	20/20	1/20	7/20
SLSA Flash, raw-strict	59/60	17/20	17/20	17/20
SLSA Flash, contract	59/60	17/20	15/20	17/20
Graph Mistral, contract	9/36	18/24	0/24	13/36
Graph Phi-4, contract	21/36	5/24	0/24	1/36

Table 7: Gatekeeper aggregate raw-strict confusion counts over three repetitions (left) and authored SLSA transition-graph contract-action confusion counts (right). N marks the endpoint's terminal failure category.

Gatekeeper

Route	Gold	ALLOW	HOLD	N
Qwen	ALLOW	4	0	32
Qwen	HOLD	0	6	30
Flash	ALLOW	32	0	4
Flash	HOLD	0	21	15

Authored transition graph

Route	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
Mistral	ALLOW	9	8	0	9	10
Mistral	HOLD	2	18	0	4	0
Mistral	DEMOTE	3	0	0	0	21
Mistral	ESCALATE	2	3	0	13	18
Phi-4	ALLOW	21	0	0	0	15
Phi-4	HOLD	1	5	0	0	18
Phi-4	DEMOTE	3	12	0	0	9
Phi-4	ESCALATE	0	0	0	1	35

Table 8: R6 and balanced-challenge full-contract confusion counts. Columns are terminal predicted actions; N is a contract failure.

Configuration	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
r6 Qwen	ALLOW	143	0	0	0	7
r6 Qwen	HOLD	4	110	0	2	18
r6 Qwen	DEMOTE	8	0	0	0	0
r6 Qwen	ESCALATE	0	0	0	5	3
Challenge Qwen	ALLOW	79	0	0	0	11
Challenge Qwen	HOLD	0	26	0	0	4
Challenge Qwen	DEMOTE	8	4	4	4	10
Challenge Qwen	ESCALATE	0	0	0	23	7
Challenge Flash	ALLOW	68	4	0	8	10
Challenge Flash	HOLD	13	9	0	0	8
Challenge Flash	DEMOTE	25	3	0	0	2
Challenge Flash	ESCALATE	0	3	0	13	14

Table 9: Balanced-challenge raw-strict action confusion counts. N marks a terminal extraction failure.

Configuration	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
Challenge Qwen	ALLOW	77	0	0	0	13
Challenge Qwen	HOLD	0	26	1	0	3
Challenge Qwen	DEMOTE	7	3	3	4	13
Challenge Qwen	ESCALATE	0	0	0	22	8
Challenge Flash	ALLOW	68	4	5	13	0
Challenge Flash	HOLD	13	10	7	0	0
Challenge Flash	DEMOTE	25	3	2	0	0
Challenge Flash	ESCALATE	0	3	2	25	0

Table 10: Fixed SLSA raw-strict and contract-action confusion counts. N is the endpoint's terminal failure category.

Route	Endpoint	Gold	ALLOW	HOLD	DEMOTE	ESCALATE	N
Qwen	raw-strict	ALLOW	60	0	0	0	0
Qwen	raw-strict	HOLD	0	20	0	0	0
Qwen	raw-strict	DEMOTE	0	0	20	0	0
Qwen	raw-strict	ESCALATE	0	0	0	20	0
Qwen	contract	ALLOW	60	0	0	0	0
Qwen	contract	HOLD	0	20	0	0	0
Qwen	contract	DEMOTE	0	0	1	0	19
Qwen	contract	ESCALATE	0	0	0	7	13
Flash	raw-strict	ALLOW	59	0	0	0	1
Flash	raw-strict	HOLD	0	17	0	0	3
Flash	raw-strict	DEMOTE	0	0	17	0	3
Flash	raw-strict	ESCALATE	0	0	0	17	3
Flash	contract	ALLOW	59	0	0	0	1
Flash	contract	HOLD	0	17	0	0	3
Flash	contract	DEMOTE	0	0	15	0	5
Flash	contract	ESCALATE	0	0	0	17	3